

PART IV**C++ STL MASTERY**

The containers and algorithms behind almost every line of contest code

The Standard Template Library is what makes C++ practical for competitive programming: instead of hand-rolling a hash table or a heap for every problem, you reach for a container whose complexity guarantees you already know cold. This Part is the single highest-leverage Part in the whole book for raw coding speed.

Contents of Part IV

- 1. Iterators and Range-Based Loops
- 2. pair and tuple
- 3. Other Sequence Containers: array, deque, list
- 4. Stack and Queue
- 5. Priority Queue (Heap)
- 6. set and multiset
- 7. map and multimap
- 8. unordered_set and unordered_map
- 9. Sorting and Custom Comparators
- 10. Binary Search Algorithms: lower_bound, upper_bound, equal_range
- 11. Everyday Algorithms: accumulate, count, find, permutations, and more
- 12. bitset
- 13. Competitive STL Combination Patterns
- Part IV Revision, Cheat Sheet, and Practice

Chapter 1 — Iterators and Range-Based Loops

1.1 What an iterator is

In one sentence: an iterator is a generalized pointer that lets you walk through any container's elements the same way, regardless of how that container stores them internally.

A vector stores elements contiguously, a map stores them in a tree, a list stores them as linked nodes — yet the same iterator syntax (`++it`, `*it`) walks through all of them. This uniform interface is exactly what lets one `sort()` function, one `find()` function, and so on, work across many different container types.

1.2 `begin/end`, `rbegin/rend`, `auto`

```
vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); it++) cout << *it << " "; //
10 20 30
for (auto it = v.rbegin(); it != v.rend(); it++) cout << *it << " ";
// 30 20 10 – reverse

for (auto x : v) cout << x << " "; // range-based loop – cleanest for
read-only
for (auto &x : v) x *= 2; // reference – modifies elements in place
```

`v.begin()` points to the first element; `v.end()` points one position past the last element (never dereference it). `auto` lets the compiler deduce an iterator's (often long, ugly) type automatically — always prefer it over spelling out the full iterator type by hand.

WARNING — Iterator invalidation

Modifying a container's size (e.g. `push_back` on a vector that must reallocate, or `erase` on most containers) can invalidate existing iterators into it — using an invalidated iterator afterward is undefined behaviour. When erasing while iterating, capture `erase()`'s return value (it returns a valid iterator to the next element) rather than incrementing a stale iterator.

```
// Safe pattern for erasing while iterating
for (auto it = v.begin(); it != v.end(); ) {
    if (*it % 2 == 0) it = v.erase(it); // erase returns the next
    valid iterator
    else it++;
}
```

Chapter 2 — pair and tuple

2.1 pair

```
pair<int, int> p = {3, 4};
cout << p.first << " " << p.second; // 3 4

vector<pair<int,int>> points = {{1,2}, {3,4}, {0,-1}};
sort(points.begin(), points.end()); // sorts by .first, then
.second, automatically
```

NOTE — pair sorts lexicographically by default

This is exactly why `vector<pair<int,int>>` is so common in competitive code: sorting by a primary key and breaking ties by a secondary key is often exactly what you want, and `pair` gives you that comparison for free, with zero extra code.

2.2 tuple

```
tuple<int, int, string> t = {5, 10, "ok"};
cout << get<0>(t) << " " << get<1>(t) << " " << get<2>(t);

auto [a, b, c] = t; // structured bindings (C++17) – unpack directly
into named variables
```

Use `tuple` when you need to group three or more values together (a pair only holds two); structured bindings make working with both pair and tuple much more readable than repeated `.first/.second/get<i>` calls.

```
for (auto &[x, y] : points) cout << x << "," << y << " "; //
structured bindings in a loop
```

Chapter 3 — Other Sequence Containers: array, deque, list

3.1 `std::array` — a fixed-size vector

```
array<int, 5> a = {1, 2, 3, 4, 5}; // size fixed at compile time,
cannot grow
```

`array` behaves like a C-style array but supports `.size()`, iterators, and works with STL algorithms directly — a good middle ground when the size truly never changes and you want the safety of STL-style syntax without vector's resizing overhead.

3.2 `deque` — double-ended queue

```
deque<int> dq;  
dq.push_back(1);  
dq.push_front(0);  
dq.pop_back();  
dq.pop_front();  
cout << dq[0]; // supports random access like a vector, too
```

A deque supports $O(1)$ insertion and removal at both ends (a vector is only $O(1)$ amortized at the back), while still allowing $O(1)$ random access via `[]`. It underlies the sliding-window-maximum technique (Part XIX) and the monotonic-deque pattern.

3.3 list — doubly linked list

```
list<int> lst = {1, 2, 3};  
lst.push_front(0);  
lst.push_back(4);  
auto it = lst.begin();  
advance(it, 2);  
lst.insert(it, 99); //  $O(1)$  insertion once you have an iterator to  
the position
```

`list` gives $O(1)$ insertion/removal at any position, given an iterator to that position — but has no random access (no `[]`); reaching the k -th element takes $O(k)$. In competitive programming, a manually-built linked list (Part VIII) is more common than `std::list` itself, but recognizing when $O(1)$ insertion-anywhere matters is the transferable idea.

Chapter 4 — Stack and Queue

4.1 stack — LIFO

```
stack<int> st;
st.push(1); st.push(2); st.push(3);
cout << st.top(); // 3 – most recently pushed
st.pop(); // removes 3 (pop() returns nothing in C++ – check top()
first if needed)
cout << st.empty(); // false while elements remain
```

LIFO — Last In, First Out. All operations (push/pop/top) are $O(1)$. Classic uses: reversing something, matching brackets/parentheses, undo functionality, and depth-first traversal implemented iteratively (Part XI, XIII).

```
// Balanced parentheses check
bool isBalanced(string &s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(') st.push(c);
        else if (c == ')') {
            if (st.empty()) return false;
            st.pop();
        }
    }
    return st.empty();
}
```

4.2 queue — FIFO

```
queue<int> q;
q.push(1); q.push(2); q.push(3);
cout << q.front(); // 1 – least recently pushed still waiting
q.pop(); // removes 1
```

FIFO — First In, First Out. Used constantly for breadth-first search (Part XIII), where you must process nodes in the order they were discovered.

4.3 Monotonic stack (a preview)

A monotonic stack keeps its elements in strictly increasing (or decreasing) order at all times, popping elements that violate the order before pushing a new one. It answers "next greater/smaller element" queries for an entire array in $O(n)$ total, instead of an $O(n^2)$ brute force:

```
vector<int> nextGreater(vector<int> &a) {
    int n = a.size();
    vector<int> result(n, -1);
    stack<int> st; // stores indices, kept so that a[st.top()] is
    decreasing bottom-to-top
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            result[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }
    return result;
}
```

KEY IDEA — Why this is $O(n)$, not $O(n^2)$

Each index is pushed onto the stack exactly once and popped at most once across the entire run — total push/pop operations are bounded by $2n$, exactly the same "bounded total movement" argument used for sliding windows in Part III.

Chapter 5 — Priority Queue (Heap)

5.1 What a priority queue gives you

A `priority_queue` always gives you the largest (by default) element in $O(1)$, and lets you insert a new element or remove the largest in $O(\log n)$. Internally it is a binary heap (Part XII covers the mechanism in full); here, focus on the interface and when to reach for it.

```
priority_queue<int> pq; // max-heap by default
pq.push(3); pq.push(1); pq.push(4);
cout << pq.top(); // 4 – the largest
pq.pop(); // removes 4

priority_queue<int, vector<int>, greater<int>> minPq; // min-heap
minPq.push(3); minPq.push(1); minPq.push(4);
cout << minPq.top(); // 1 – the smallest
```

5.2 priority_queue of pairs, and custom comparators

```
priority_queue<pair<int,int>> pq; // orders by .first, ties broken
by .second, both descending
pq.push({5, 100});
pq.push({5, 50});
pq.push({9, 1});
cout << pq.top().first; // 9

// Custom comparator via a struct (needed when the ordering isn't a
simple pair/tuple comparison)
struct Compare {
    bool operator()(pair<int,int> &a, pair<int,int> &b) {
        return a.second > b.second; // smaller .second has higher
priority
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Compare> pq2;
```

NOTE — Where priority_queue shows up constantly

Dijkstra's shortest path (Part XIII), any "process the k smallest/largest so far" problem, and merging multiple sorted lists all reach for a priority queue as the natural structure — it's worth being fluent with the custom-comparator syntax specifically because Dijkstra needs a min-heap of (distance, node) pairs.

Chapter 6 — set and multiset

6.1 set — sorted, unique elements

```
set<int> s;  
s.insert(5); s.insert(1); s.insert(3); s.insert(1); // duplicate  
insert is a no-op  
for (int x : s) cout << x << " "; // 1 3 5 – always in sorted order  
  
cout << s.count(3); // 1 (present) or 0 (absent) – never more, since  
elements are unique  
s.erase(3);  
auto it = s.find(1); // returns an iterator, or s.end() if not found  
if (it != s.end()) cout << "found";
```

Internally, set is a balanced binary search tree (typically a red-black tree), which is exactly why it stays sorted automatically and why every operation (insert, erase, find, count) costs $O(\log n)$.

6.2 Order-statistics operations: lower_bound and upper_bound on a set

```
set<int> s = {10, 20, 30, 40};  
auto it1 = s.lower_bound(25); // iterator to first element >= 25 ->  
30  
auto it2 = s.upper_bound(30); // iterator to first element > 30 ->  
40  
cout << *it1 << " " << *it2; // 30 40
```

WARNING — Use the member function, not the free function, on a set

There are two versions of `lower_bound`: the standalone `<algorithm>` version (Chapter 10) works on any sorted range in $O(\log n)$ only if the container supports random-access iterators, but a `set`'s iterators are not random-access — calling the free function on a `set` silently degrades to $O(n)$. Always call `s.lower_bound(x)` as a member function on a `set/map` to keep the $O(\log n)$ guarantee.

6.3 multiset — sorted, duplicates allowed

```
multiset<int> ms;
ms.insert(5); ms.insert(5); ms.insert(3);
cout << ms.count(5); // 2

// Erasing ALL copies of a value vs. exactly one copy
ms.erase(5); // removes every 5 currently in the multiset!
ms.insert(5); ms.insert(5);
ms.erase(ms.find(5)); // removes exactly ONE 5, using an iterator
```

WARNING — The most common multiset bug

`ms.erase(value)` removes every occurrence of that value, not just one — if you want to remove a single occurrence, erase via an iterator: `ms.erase(ms.find(value))`.

Chapter 7 — map and multimap

7.1 map — sorted key-value store

```
map<string, int> ageOf;
ageOf["Alice"] = 30;
ageOf["Bob"] = 25;
cout << ageOf["Alice"]; // 30
cout << ageOf["Charlie"]; // 0 – accessing a missing key CREATES it
with a default value!

if (ageOf.count("Dave") == 0) cout << "Dave not present"; // safe
existence check
for (auto &[name, age] : ageOf) cout << name << ":" << age << " "; //
iterates in sorted key order
```

WARNING — operator[] silently inserts

Reading `m[key]` for a key that doesn't exist yet inserts it with a default-constructed value (0 for `int`) as a side effect. If you only want to check for presence without modifying the map, use `m.count(key)` or `m.find(key)` instead of `m[key]`.

7.2 Complexity and why map iterates in sorted order

Like `set`, `map` is backed by a balanced BST keyed on its keys — so `insert`, `erase`, `find`, and `operator[]` are all $O(\log n)$, and iterating the whole map visits keys in ascending order automatically, at no extra cost.

7.3 map, int> and other compound keys

```
map<pair<int,int>, int> dist; // e.g. distance between two labeled
points
dist[{0, 1}] = 5;
dist[{1, 2}] = 3;
cout << dist[{0, 1}]; // 5
```

Any type with a well-defined ordering (like `pair`, which orders lexicographically — Chapter 2) can be a map's key directly, no extra

code required.

7.4 multimap

```
multimap<string, int> scores;
scores.insert({"Alice", 90});
scores.insert({"Alice", 85}); // Alice can have multiple entries
auto range = scores.equal_range("Alice");
for (auto it = range.first; it != range.second; it++) cout <<
it->second << " "; // 85 90
```

NOTE — multimap has no operator[]

Because a key can map to several values, multimap doesn't support `m[key]` at all — use `insert` and `equal_range` as shown above.

Chapter 8 — unordered_set and unordered_map

8.1 What "unordered" and "hashed" mean

`unordered_set` and `unordered_map` use a hash table internally instead of a balanced tree: each key is passed through a hash function that produces a bucket index, and the key is stored in that bucket. This trades away sorted iteration order for a large speed gain: operations are $O(1)$ on average, versus $O(\log n)$ for `set/map`.

```
unordered_map<int, int> freq;
for (int x : a) freq[x]++;

unordered_set<int> seen;
seen.insert(5);
cout << seen.count(5); // 0(1) average
```

8.2 Why "O(1) average" is not "O(1) always"

If many keys hash into the same bucket (a collision), that bucket degrades toward a linear list, and operations on it become $O(\text{bucket size})$ rather than $O(1)$. With a good hash function and typical random data this essentially never happens — but on adversarially chosen input (deliberately crafted to collide against a known hash implementation), or with a poor custom hash, `unordered_map`'s worst case degrades to $O(n)$ per operation.

WARNING — Codeforces anti-hash-hacking

Codeforces problems have historically included test cases specifically designed to break naive `unordered_map<int,int>` usage via hash collisions, causing sudden, severe TLE. A common defense is adding a random offset via a custom hash function, or simply preferring `map` when n isn't large enough for the $O(\log n)$ vs $O(1)$ difference to matter.

8.3 When to choose ordered vs. unordered

Need	Choose
Fastest average lookup/insert, don't care about order	<code>unordered_set</code> / <code>unordered_map</code>
Need sorted iteration, or <code>lower_bound</code> / <code>upper_bound</code> queries	<code>set</code> / <code>map</code>
Worried about adversarial hash-collision test cases	<code>map</code> / <code>set</code> , or <code>unordered_</code> with a custom hash
Keys are small bounded integers	Often a plain array is faster than either

KEY IDEA — Chapters 6-8 quick revision

set/map: $O(\log n)$ operations, sorted iteration, backed by a balanced BST.

unordered_set/unordered_map: $O(1)$ average operations, no ordering guarantee, backed by a hash table.

m[key] on a map inserts a default value as a side effect if the key is missing — use count()/find() for a pure existence check.

multiset::erase(value) removes ALL copies — erase by iterator to remove exactly one.

Chapter 9 — Sorting and Custom Comparators

9.1 sort() and stable_sort()

```
vector<int> v = {5, 2, 8, 1};  
sort(v.begin(), v.end()); // ascending,  $O(n \log n)$   
sort(v.begin(), v.end(), greater<int>()); // descending  
  
sort(v.rbegin(), v.rend()); // another way to sort descending, via  
reverse iterators
```

sort() is not guaranteed stable (equal elements may be reordered relative to each other) and typically runs an introsort (quicksort/heapsort hybrid) in $O(n \log n)$. stable_sort() guarantees equal elements keep their original relative order, useful when sorting by one key while relying on a previous sort by another key, at the cost of slightly more memory (it uses merge sort internally, Part VI).

9.2 Lambda comparators

```
vector<pair<int,int>> people = {{25, 1}, {30, 0}, {25, 0}};
// Sort by age ascending, then by original index descending
sort(people.begin(), people.end(), [](pair<int,int> &a,
pair<int,int> &b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
}));
```

WARNING — A comparator must define a strict weak ordering

A comparator function must return false when comparing an element to itself, and must be consistent (if it says $a < b$, it must not also say $b < a$). A comparator using $\leq$ instead of $<$ violates this and causes undefined behaviour (often a crash or an infinite loop) inside `sort()` — this is a subtle, easy-to-write bug worth being deliberate about.

9.3 Struct comparators

```
struct ByLengthThenAlpha {
    bool operator()(const string &a, const string &b) const {
        if (a.size() != b.size()) return a.size() < b.size();
        return a < b;
    }
};
sort(words.begin(), words.end(), ByLengthThenAlpha());
```

A struct comparator is preferred over a lambda when the same comparator is reused in several places (e.g. both a `sort()` call and a set's template parameter), since a lambda's type can't easily be named twice.

9.4 Custom sort key: sorting indices instead of values

A very common competitive pattern: you need the sorted order of an array's elements (their original indices), not the sorted values themselves.

```
vector<int> a = {40, 10, 30, 20};
vector<int> idx(a.size());
iota(idx.begin(), idx.end(), 0); // idx = {0, 1, 2, 3}
sort(idx.begin(), idx.end(), [&](int i, int j) { return a[i] < a[j];
});
// idx is now {1, 3, 2, 0} – the original positions, in the order
their values would sort to
```

Chapter 10 — Binary Search Algorithms: `lower_bound`, `upper_bound`, `equal_range`

10.1 `binary_search` — existence only

```
vector<int> v = {1, 3, 3, 5, 7};
cout << binary_search(v.begin(), v.end(), 3); // 1 (true) – but
tells you nothing about WHERE
```

10.2 `lower_bound` and `upper_bound`

Both require the range to already be sorted. `lower_bound(first, last, x)` returns an iterator to the first element that is not less than x (i.e. $\geq x$). `upper_bound(first, last, x)` returns an iterator to the first element that is strictly greater than x . Both run in $O(\log n)$.

```
vector<int> v = {1, 3, 3, 3, 5, 7};
auto lo = lower_bound(v.begin(), v.end(), 3); // points to the FIRST
3, index 1
auto hi = upper_bound(v.begin(), v.end(), 3); // points to the 5,
index 4
int count3 = hi - lo; // 3 – number of times 3 appears
```

Common uses table

Goal	Expression
First occurrence of x	<code>lower_bound(begin, end, x)</code>
Last occurrence of x	<code>prev(upper_bound(begin, end, x))</code>
Count of x in a sorted range	<code>upper_bound(...) - lower_bound(...)</code>
Does x exist?	<code>lower_bound(...) != end && *lower_bound(...) == x</code>
Smallest element strictly greater than x	<code>upper_bound(begin, end, x)</code>
Largest element less than or equal to x	<code>prev(upper_bound(begin, end, x))</code> , if it exists

10.3 equal_range

```
auto range = equal_range(v.begin(), v.end(), 3); // {lower_bound,
upper_bound} in one call
cout << (range.second - range.first); // count of 3s, same result as
above
```

10.4 Binary search on the answer (a first preview)

The same idea generalizes beyond searching a literal sorted array: if a yes/no property is monotonic (once it becomes true for some value, it stays true for all larger — or all smaller — values), you can binary-search directly on the answer itself, not on array contents. Part VI develops this pattern in full; recognizing the shape now avoids treating it as unrelated later.

Chapter 11 — Everyday Algorithms

11.1 accumulate, count, count_if, find, find_if

```
vector<int> v = {1, 2, 3, 4, 5};
long long sum = accumulate(v.begin(), v.end(), 0LL); // 15
int cnt = count(v.begin(), v.end(), 3); // 1
int evenCnt = count_if(v.begin(), v.end(), [](int x){ return x % 2
== 0; }); // 2
auto it = find(v.begin(), v.end(), 3); // iterator to the 3
auto it2 = find_if(v.begin(), v.end(), [](int x){ return x > 3; });
// iterator to the 4
```

WARNING — accumulate's starting value type matters

`accumulate(begin, end, 0)` accumulates as `int` because the initial value `0` is an `int` — even if the vector holds values that would overflow `int` when summed. Use `0LL` to force the accumulation to happen in `long long`.

11.2 min, max, min_element, max_element

```
int m = min(3, 7); // 3
int mx = max({3, 7, 1, 9, 2}); // 9 – works on an initializer list
too
auto it = min_element(v.begin(), v.end());
cout << *it << " at index " << (it - v.begin());
```

11.3 reverse and rotate

```
reverse(v.begin(), v.end()); // reverses in place, O(n)
rotate(v.begin(), v.begin() + 2, v.end()); // rotates so that
element at index 2 becomes the new first
```

NOTE — rotate replaces Part III's manual reversal trick

The rotation-by-three-reversals technique from Part III is exactly what `rotate()` does internally — knowing the manual version matters for understanding why it works and for languages/situations without a built-in; in contest C++, prefer calling `rotate()` directly.

11.4 `next_permutation` and `prev_permutation`

```
vector<int> v = {1, 2, 3};
do {
    for (int x : v) cout << x << " ";
    cout << "\n";
} while (next_permutation(v.begin(), v.end()));
// prints all 6 permutations of {1,2,3} in lexicographic order
```

WARNING — Must start sorted for a complete cycle

`next_permutation` generates the next permutation in lexicographic order and returns false when it wraps back around to the smallest. To enumerate every permutation, the vector must start sorted ascending — starting from an arbitrary order only generates permutations from that point onward, not all of them.

11.5 `swap`

```
swap(a, b); // swaps two variables of the same type
swap(v[i], v[j]); // swaps two elements of a container
```

Chapter 12 — `bitset`

12.1 What `bitset` gives you

A `bitset<N>` stores exactly `N` bits, packed tightly (8 bits per byte, unlike `vector<bool>`'s more awkward specialization), and supports bitwise operations on the entire set of bits at once, extremely fast — a whole `bitset<64>` AND/OR/XOR operation compiles down to a handful of CPU instructions rather than 64 separate ones.

```
bitset<8> b1("10110010");
bitset<8> b2(19); // constructed from an integer
cout << b1; // 10110010
cout << b1.count(); // 4 – number of set bits
cout << b1[2]; // reads bit at position 2
b1.set(0); // sets bit 0 to 1
b1.reset(1); // sets bit 1 to 0
b1.flip(); // flips every bit
cout << (b1 & b2); // bitwise AND across all bits at once
```

NOTE — Where `bitset` earns its place

Problems needing a large boolean array with fast bulk operations — reachability tracking, subset-sum feasibility over a bounded range, or speeding up a DP that's fundamentally about which values are achievable — often gain a large constant-factor speedup by switching from `vector<bool>` or a plain boolean array to `bitset`.

Chapter 13 — Competitive STL Combination Patterns

13.1 A reference table of patterns you will see constantly

Pattern	Typical use
<code>vector<></code>	Weighted edges, coordinates, (value, index) pairs for custom sorting
<code>map / unordered_map</code>	Frequency counting, compressing large values to small IDs
<code>set / multiset</code>	Maintaining a dynamic sorted collection with fast insert/erase/lookup
<code>priority_queue<></code>	Dijkstra's algorithm: (distance, node), using a min-heap variant
<code>vector<></code>	Grids, and graph adjacency lists
<code>unordered_map</code>	Counting or ID-mapping distinct strings (word frequency, compression)
<code>map,int></code>	State-keyed values, e.g. memoized results indexed by a coordinate pair

13.2 Worked pattern: coordinate compression

When values are large (e.g. up to 10^9) but you only care about their relative order, mapping them to small dense indices (0, 1, 2, ...) lets you use array-based structures (frequency arrays, Fenwick trees — Part XVIII) instead of hash maps.

```
vector<int> vals = {100, 5000000, 42, 5000000, 7};
vector<int> sorted_vals = vals;
sort(sorted_vals.begin(), sorted_vals.end());
sorted_vals.erase(unique(sorted_vals.begin(), sorted_vals.end()),
sorted_vals.end());

auto compress = [&](int x) {
    return lower_bound(sorted_vals.begin(), sorted_vals.end(), x) -
sorted_vals.begin();
};
for (int x : vals) cout << compress(x) << " "; // each value replaced
by its rank, 0-indexed
```

This combines `sort`, `unique` (which moves duplicates to the end and returns a new logical end — Chapter 11's pattern of "algorithm returns an iterator, follow it with `erase` to actually shrink the container"), and `lower_bound` into one clean, very common technique.

KEY IDEA — Chapter 13 quick revision

Recognizing these container combinations on sight is a major speed advantage under contest time pressure — they show up far more often than any single 'clever trick'.

Coordinate compression = `sort` + `unique/erase` + `lower_bound`, used whenever large values only matter by relative order.

Part IV — Master Revision

Must Remember

- `set/map`: $O(\log n)$ operations, sorted order, backed by a balanced BST.
- `unordered_set/unordered_map`: $O(1)$ average operations, no order guarantee, backed by a hash table — worst case $O(n)$ on adversarial input.
- `map`'s `operator[]` inserts a default value as a side effect on a missing key — use `count()/find()` for a pure existence check.
- `multiset::erase(value)` removes every occurrence; `erase` by iterator to remove exactly one.
- `lower_bound` gives the first element $\geq x$; `upper_bound` gives the first element $> x$; both require a sorted range and run in $O(\log n)$.
- On a `set/map` specifically, call `.lower_bound()/upper_bound()` as member functions to keep $O(\log n)$ — the free-function versions degrade to $O(n)$ on non-random-access iterators.
- A sort comparator must be a strict weak ordering — never use `<=` in place of `<`.
- `priority_queue` is a max-heap by default; pass `greater()` (or a custom comparator) for a min-heap.

Complexity Table — Part IV

Container / Operation	Complexity
vector push_back (amortized) / [] access	O(1)
deque push_front/push_back / [] access	O(1)
stack/queue push/pop/top/front	O(1)
priority_queue push/pop	O(log n)
set/map insert/erase/find/count	O(log n)
unordered_set/unordered_map insert/erase/find (average)	O(1) average, O(n) worst case
sort()	O(n log n)
binary_search / lower_bound / upper_bound (sorted range)	O(log n)
accumulate / count / find / min_element / max_element	O(n)
next_permutation (single call, amortized over a full cycle)	O(n) amortized

Common Mistakes Recap

Mistake	Fix
Using <code>m[key]</code> just to check existence	Use <code>m.count(key)</code> or <code>m.find(key)</code> instead
<code>ms.erase(value)</code> to remove one copy from a multiset	Use <code>ms.erase(ms.find(value))</code>
Comparator using <code><=</code> instead of <code><</code>	Always use strict <code><</code> for a valid strict weak ordering
<code>accumulate(v.begin(), v.end(), 0)</code> on values that overflow <code>int</code>	Use <code>0LL</code> as the starting value
Calling the free <code>lower_bound()</code> on a set/map	Call it as a member function: <code>s.lower_bound(x)</code>
Assuming <code>unordered_map</code> is always $O(1)$	Remember the adversarial worst case; prefer <code>map</code> if unsure and <code>n</code> is small enough
Calling <code>next_permutation</code> on an unsorted starting vector expecting all permutations	Sort ascending first

Interview-Style Questions

- 1 Why is `unordered_map` usually $O(1)$ on average, and under what circumstances can it degrade to $O(n)$?
- 2 What is the practical difference between `lower_bound` and `upper_bound`?
- 3 Why does `map` iterate its keys in sorted order but `unordered_map` does not?
- 4 What happens if you access a missing key with `map's operator[]`, and why can that be a subtle bug?
- 5 Why must a sort comparator implement a strict weak ordering, and what goes wrong if it doesn't?

Cheat Sheet — Part IV

```
// Frequency counting
unordered_map<int,int> freq;
for (int x : a) freq[x]++;

// Min-heap for Dijkstra-style algorithms
priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;

// Coordinate compression
vector<int> sv = vals;
sort(sv.begin(), sv.end());
sv.erase(unique(sv.begin(), sv.end()), sv.end());
int rank = lower_bound(sv.begin(), sv.end(), x) - sv.begin();

// Count occurrences of x in a sorted vector v
int cnt = upper_bound(v.begin(), v.end(), x) -
lower_bound(v.begin(), v.end(), x);

// Custom sort: ascending by .first, descending by .second
sort(v.begin(), v.end(), [](auto &a, auto &b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
});
```

Problem P4.1 — Top-k frequent elements [\[Level 2\]](#)

Given an array, return the k most frequent elements.

Hint 1: Count frequencies with `unordered_map`, then think about which container gives you the k largest frequencies efficiently.

Key observation: A min-heap of size k (`priority_queue` with `greater<>>`) that pops whenever it exceeds size k keeps exactly the k largest-frequency elements seen so far.

Complexity: $O(n)$ for counting + $O(d \log k)$ for the heap step, where d is the number of distinct elements.

Problem P4.2 — First unique character [Level 1]

Given a string, find the index of the first character that does not repeat anywhere else in the string.

Hint 1: A single frequency pass, then a second pass checking each character's count.

Complexity: $O(n)$ time using a 26-length frequency array (Part III), $O(1)$ extra space.

NOTE — Coming up in Part V

Part V — Complexity formalizes the Big-O intuition built up across Parts I-IV into a rigorous toolkit: analyzing recursion, STL operation costs, amortized analysis in full, and best/average/worst case — everything needed to reason confidently about any algorithm's speed before you finish writing it.